

Requirement Analysis

Technology Stack (Architecture & Stack)

Date	14-07-2026
Team ID	
Project Name	EduGenie
Maximum Marks	

Technical Architecture:

EduGenie follows a simple, lightweight two-tier architecture: a responsive frontend that captures learner input and displays AI-generated output, and a modular FastAPI backend that routes requests to the appropriate module and communicates with a generative AI service.

[Learner] → [Frontend: HTML/CSS/JS] → [FastAPI Backend Router] → [Q&A / Explanation / Quiz / Summarizer / Learning Path Module] → [Generative AI Service] → back to [Frontend] → [Learner]

Table-1: Components & Technologies:

S.No	Component	Description	Technology
1.	User Interface	How the learner interacts with EduGenie	HTML, CSS, JavaScript
2.	Application Logic	Core backend logic: request routing, validation, prompt construction	Python (FastAPI)
3.	AI Integration Layer	Handles Q&A, explanation, quiz, summary, and learning-path generation	Google Gemini API (cloud-based generative AI)
4.	Local Model Support (optional)	Lightweight local inference for basic tasks to reduce cloud dependency	Lightweight local LLM (optional, Mac M1 optimized)
5.	Session/Context Storage	Maintains conversational context within a session	In-memory session store (no heavy database required for MVP)
6.	File/Text Input Handling	Accepts pasted text for the summarization module	FastAPI request body / form handling
7.	External API	Provides the underlying language understanding and generation	Generative AI provider REST API
8.	Infrastructure	Application deployment target	Local server, optimized for resource-constrained hardware (e.g., Mac M1)

Table-2: Application Characteristics:

S.No	Characteristics	Description	Technology
1.	Open-Source Frameworks	Core frameworks used to build the application	FastAPI, Uvicorn
2.	Security	Access control and credential handling	Environment-variable based API key storage; no keys exposed to frontend
3.	Scalable Architecture	Modular router/service design per feature	FastAPI routers with independent service modules

4.	Availability	Ensuring core features remain accessible	Graceful error handling and fallback messaging on AI service failure
5.	Performance	Design considerations for responsiveness on constrained hardware	Lightweight dependencies, minimal frontend bundle, efficient async request handling